

# TCGA–DSMZ Correlation Analysis

chu25

2025-09-15

```
config <- list(                                     # Define configuration list for pipeline
  # Inputs
  tcga_se_rds   = "/home/chu25/data/tcga/ALL_TCGA_STAR_Counts_SummarizedExperiment_filtered.rds",
  dsmz_rds      = "/home/chu25/data/dsmz/DSMZ_count_gene.rds", # Path to DSMZ count data RDS
  dsmz_meta_csv = "/home/chu25/data/dsmz/DSMZ_metadata.csv",   # Path to DSMZ metadata CSV
  purity_tsv    = NULL,                                       # Optional path to purity TSV (sample, purity)

  # Outputs
  outdir        = "/home/chu25/dsmz/results/correlation4", # Output directory for results

  # Analysis
  var_gene_set  = "tcga_5k", # Gene set for variable gene selection
  min_shared    = 1000,      # Minimum number of shared genes required
  batch_adjust_method = "combat_seq" # Batch correction method ("none", "combat")
)

dir.create(config$outdir, showWarnings = FALSE, recursive = TRUE) # Create output directory,
knitr::opts_chunk$set(fig.path = paste0(config$outdir, "/figs-")) # Set figure output path for

# =====
# DATA LOADING
# =====

# Load TCGA SummarizedExperiment and extract counts
load_tcga_data <- function(file_path) {
  cat("[INFO] Loading TCGA data...\n") # Print message indicating TCGA data loading
  if (!file.exists(file_path)) stop(sprintf("[ERROR] TCGA RDS not found: %s", file_path)) # Stop if file not found
  se <- readRDS(file_path) # Read TCGA SummarizedExperiment from RDS
  counts <- assay(se) # Extract count matrix from SummarizedExperiment
  cat(sprintf("[INFO] TCGA dims: %d genes x %d samples\n", nrow(counts), ncol(counts))) # Print dimensions
  list(se = se, counts = counts) # Return list with SummarizedExperiment and counts
}

# Load DSMZ raw wide table (counts + gene columns) and metadata CSV
load_dsmz_data <- function(counts_path, meta_path) {
  cat("[INFO] Loading DSMZ data...\n") # Print message indicating DSMZ data loading
```

```

if (!file.exists(counts_path)) stop(sprintf("[ERROR] DSMZ RDS not found: %s", counts_path))
if (!file.exists(meta_path)) stop(sprintf("[ERROR] DSMZ metadata CSV not found: %s", meta_path))
raw <- readRDS(counts_path) # Read DSMZ raw count data from RDS
meta <- read.csv(meta_path) # Read DSMZ metadata from CSV
stopifnot(all(c("Ensembl_ID", "gene_name") %in% colnames(raw))) # Verify required columns in raw
stopifnot("sample_name" %in% names(meta)) # Verify sample_name column in metadata
cat(sprintf("[INFO] DSMZ table: %d rows x %d cols\n", nrow(raw), ncol(raw))) # Print dimensions
list(raw = raw, meta = meta) # Return list with raw counts and metadata
}

# =====
# GENE ID HARMONIZATION
# =====

# Remove Ensembl version suffixes and sum duplicates
harmonize_gene_ids <- function(tcga_counts) { # Define function to harmonize TCGA gene IDs
  cat("[INFO] Harmonizing TCGA gene IDs...\n") # Print message indicating gene ID harmonization
  g <- sub("\\..*$", "", rownames(tcga_counts)) # Remove version suffixes from Ensembl IDs
  if (any(duplicated(g))) { # Check for duplicate gene IDs
    tcga_counts <- rowsum(tcga_counts, g, reorder = TRUE) # Sum counts for duplicate genes
    rownames(tcga_counts) <- sort(unique(g)) # Set unique sorted gene IDs as row names
  } else {
    rownames(tcga_counts) <- g # Set harmonized gene IDs as row names
  }
  tcga_counts # Return harmonized count matrix
}

# Build a numeric DSMZ count matrix from the raw wide table
build_dsmz_matrix <- function(dsmz_raw) { # Define function to build DSMZ count matrix
  cat("[INFO] Building DSMZ count matrix...\n") # Print message indicating matrix construction
  annot <- c("Ensembl_ID", "gene_name", "Ensembl_ID_with_version") # Define annotation columns
  sample_cols <- setdiff(colnames(dsmz_raw), annot) # Identify sample columns
  dsmz_raw[sample_cols] <- lapply(dsmz_raw[sample_cols], function(x) as.numeric(as.character(x))) # Convert sample columns to numeric
  M <- as.matrix(dsmz_raw[, sample_cols, drop=FALSE]) # Create matrix from sample columns
  rownames(M) <- dsmz_raw$Ensembl_ID # Set Ensembl IDs as row names
  if (any(duplicated(rownames(M)))) # Check for duplicate gene IDs
    M <- rowsum(M, rownames(M), reorder = TRUE) # Sum counts for duplicate genes
  keep <- vapply(as.data.frame(M), function(v) any(!is.na(v)), logical(1)) # Identify columns with non-NA values
  if (!all(keep)) { # Check if any columns are all NA
    cat(sprintf("[WARN] Dropping %d DSMZ columns that are all NA\n", sum(!keep))) # Warn about all-NA columns
    M <- M[, keep, drop=FALSE] # Drop all-NA columns
  }
  M # Return DSMZ count matrix
}

# Align DSMZ metadata to count matrix; write out unmatched for audit
align_metadata <- function(dsmz_meta, dsmz_counts, outdir) { # Define function to align metadata

```

```

cat("[INFO] Aligning metadata to counts...\n")      # Print message indicating alignment
dsmz_meta <- dsmz_meta %>% mutate(sample_id = sample_name) # Create sample_id column from sample_name
matched <- intersect(dsmz_meta$sample_id, colnames(dsmz_counts)) # Find common samples between metadata and counts
md_only <- setdiff(dsmz_meta$sample_id, colnames(dsmz_counts)) # Find metadata samples not in counts
ct_only <- setdiff(colnames(dsmz_counts), dsmz_meta$sample_id) # Find count samples not in metadata
if (length(md_only)) # Check if there are unmatched metadata samples
  write.csv(tibble(sample_name = md_only), # Save unmatched metadata samples to CSV
            file.path(outdir, "unmatched_metadata_samples.csv"),
            row.names = FALSE)
if (length(ct_only)) # Check if there are unmatched count samples
  write.csv(tibble(sample_name = ct_only), # Save unmatched count samples to CSV
            file.path(outdir, "unmatched_count_columns.csv"),
            row.names = FALSE)
dsmz_meta <- dsmz_meta %>% filter(sample_id %in% matched) # Filter metadata to matched samples
dsmz_counts <- dsmz_counts[, dsmz_meta$sample_id, drop=FALSE] # Subset counts to matched samples
stopifnot(identical(colnames(dsmz_counts), dsmz_meta$sample_id)) # Verify counts and metadata match
cat(sprintf("[INFO] Matched: %d | Unmatched(meta): %d | Unmatched(counts): %d\n",
            length(matched), length(md_only), length(ct_only))) # Print alignment summary
list(meta = dsmz_meta, counts = dsmz_counts) # Return list with aligned metadata and counts
}

```

```

# =====
# PURITY HANDLING (LOG SCALE)
# =====

# Load tumor purity vector from SummarizedExperiment or external TSV
load_purity_data <- function(tcga_se, purity_file=NULL) { # Define function to load tumor purity data
  p <- NULL # Initialize purity vector as NULL
  if ("purity" %in% colnames(colData(tcga_se))) { # Check if purity data is in SummarizedExperiment
    p <- as.numeric(colData(tcga_se)$purity) # Extract purity values as numeric
    names(p) <- colnames(assay(tcga_se)) # Assign sample names to purity values
    cat("[INFO] Using purity from SummarizedExperiment\n") # Print message indicating source
  } else if (!is.null(purity_file) && file.exists(purity_file)) { # Check if external purity file exists
    tab <- read.delim(purity_file, stringsAsFactors=FALSE) # Read purity TSV file
    stopifnot(all(c("sample", "purity") %in% colnames(tab))) # Verify required columns in TSV
    p <- setNames(tab$purity, tab$sample) # Create named purity vector
    cat("[INFO] Using external purity file\n") # Print message indicating source
  } else {
    cat("[INFO] No purity data available\n") # Print message if no purity data
  }
  p # Return purity vector
}

# Adjust TCGA LOG matrix for purity: (1) drop negative-purity-correlated genes, (2) regress out purity
# NOTE: operates on log2-CPM (not raw counts)
purity_adjust_log <- function(tcga_log, purity) { # Define function to adjust TCGA log2-CPM for purity
  if (is.null(purity)) return(tcga_log) # Return input if no purity data
}

```

```

cat("[INFO] Applying purity adjustment on log2-CPM...\n") # Print message indicating adjust
keep_samp <- intersect(colnames(tcga_log), names(purity)[!is.na(purity)]) # Find samples wi
M <- tcga_log[, keep_samp, drop=FALSE] # Subset log matrix to valid samples
pu <- purity[keep_samp] # Subset purity to valid samples
if (ncol(M) < 3) { # Check if enough samples for adjustment
  cat("[WARN] Too few samples with purity to adjust; returning input.\n") # Warn if insuffi
  return(tcga_log) # Return input matrix
}
# Identify & drop genes strongly NEG correlated with purity (likely infiltration)
ct <- apply(M, 1, function(v) suppressWarnings(cor.test(as.numeric(v), pu, method="spearman")
pv <- vapply(ct, `[`, numeric(1), "p.value") # Extract p-values
rh <- vapply(ct, function(x) unname(x$estimate), numeric(1)) # Extract correlation coeffici
padj <- p.adjust(pv, "BH") # Adjust p-values for multiple testing
drop <- names(which(padj < 0.01 & rh < -0.4)) # Identify genes with significant negat
if (length(drop)) { # Check if genes need to be dropped
  cat(sprintf("[INFO] Removing %d purity-associated genes\n", length(drop))) # Print number
  M <- M[setdiff(rownames(M), drop), , drop=FALSE] # Remove identified genes
}
# Regress out infiltration (1 - purity) on the log scale
infiltr <- 1 - pu # Calculate infiltration (1 - purity)
design <- model.matrix(~ infiltr) # Create design matrix for regression
fit <- lmFit(M, design) # Fit linear model using limma
beta <- fit$coefficients[, "infiltr", drop=FALSE] # Extract infiltration coefficients
Madj <- as.matrix(M) - beta %*% t(infiltr) # Adjust matrix by subtracting infiltra
out <- tcga_log # Initialize output matrix
common_g <- intersect(rownames(Madj), rownames(out)) # Find common genes
out[common_g, colnames(Madj)] <- Madj[common_g, ] # Update adjusted values
out # Return adjusted matrix
}

```

```

# =====
# CORRELATION + MAPPING
# =====

# Compute TCGA cohort means (on whatever scale tcga_mat is; here it's log2-CPM)
compute_tcga_means <- function(tcga_se, tcga_mat) { # Define function to compute TCGA cohort
  nm <- intersect(c("project_id", "study", "disease_type"), colnames(colData(tcga_se))) # Se
  stopifnot(!is.na(nm)) # Verify metadata field exists
  labs <- as.character(colData(tcga_se)[colnames(tcga_mat), nm]) # Extract cohort labels
  stopifnot(!any(is.na(labs))) # Verify no missing labels
  labs <- sub("^TCGA-", "", labs) # Remove "TCGA-" prefix from labels
  lev <- sort(unique(labs)) # Get sorted unique cohort labels
  means <- sapply(lev, function(ct) rowMeans(tcga_mat[, labs==ct, drop=FALSE], na.rm=TRUE)) #
  colnames(means) <- lev # Set cohort names as column names
  list(means=means, labels=setNames(labs, colnames(tcga_mat))) # Return means and labels
}

```

```

# Select variable genes by IQR
select_variable_genes <- function(mat, var_gene_set) { # Define function to select variable genes
  sel <- rownames(mat) # Default to all genes
  if (tolower(var_gene_set) %in% c("tcga_5k","tcga_10k")) { # Check if specific gene set is r
    n <- if (tolower(var_gene_set)=="tcga_5k") 5000 else 10000 # Set number of genes (5k or 1
    iqr <- matrixStats::rowIQRs(mat) # Calculate interquartile range for each
    n <- min(n, length(iqr)) # Limit to available genes
    sel <- names(sort(setNames(iqr, rownames(mat)), decreasing=TRUE))[seq_len(n)] # Select top
  }
  cat(sprintf("[INFO] Variable-gene setting: %s (%d genes used)\n", var_gene_set, length(sel)))
  sel # Return selected gene names
}

# Spearman correlations between DSMZ samples and TCGA cohort means (same gene set & scale)
compute_correlations <- function(dsmz_mat, tcga_means, dsmz_meta, sel_genes) { # Define functi
  cat("[INFO] Computing Spearman correlations...\n") # Print message indicating correlation
  spearman <- outer( # Compute correlations using outer product
    dsmz_meta$sample_id, # DSMZ sample IDs
    colnames(tcga_means), # TCGA cohort names
    Vectorize(function(cell, cohort) { # Define correlation function
      suppressWarnings( # Suppress warnings (e.g., for NA handling)
        cor( # Compute Spearman correlation
          dsmz_mat[sel_genes, cell], # DSMZ sample data
          tcga_means[sel_genes, cohort], # TCGA cohort mean data
          method = "spearman", # Use Spearman method
          use = "pairwise.complete.obs" # Handle missing values pairwise
        )
      )
    })
  )
  dimnames(spearman) <- list(dsmz_meta$sample_id, colnames(tcga_means)) # Set matrix dimension
  spearman # Return correlation matrix
}

# Organ→TCGA mapping
get_organ_tcga_mapping <- function() { # Define function to map organs to TCGA
  list( # Return list of organ-to-TCGA mappings
    "Adrenal" = c("ACC","PCPG"), # Adrenal cancers
    "Bladder" = "BLCA", # Bladder cancer
    "Brain/CNS" = c("GBM","LGG"), # Brain/CNS cancers
    "Breast" = "BRCA", # Breast cancer
    "Cervix" = "CESC", # Cervical cancer
    "Liver/Biliary" = c("LIHC","CHOL"), # Liver and biliary cancers
    "Colon/Rectum" = c("COAD","READ"), # Colorectal cancers
    "Esophagus" = "ESCA", # Esophageal cancer
    "Head/Neck" = "HNSC", # Head and neck cancer
    "Kidney" = c("KICH","KIRC","KIRP"), # Kidney cancers
  )
}

```

```

"Lung"          = c("LUAD", "LUSC"),          # Lung cancers
"Mesothelium"   = "MESO",                     # Mesothelioma
"Ovary"         = "OV",                       # Ovarian cancer
"Pancreas"      = "PAAD",                     # Pancreatic cancer
"Prostate"      = "PRAD",                     # Prostate cancer
"Sarcoma"       = "SARC",                     # Sarcoma
"Skin"          = "SKCM",                     # Skin cancer
"Stomach"       = "STAD",                     # Stomach cancer
"Testis"        = "TGCT",                     # Testicular cancer
"Thyroid"       = "THCA",                     # Thyroid cancer
"Thymus"        = "THYM",                     # Thymic cancer
"Uterus"        = c("UCEC", "UCS"),           # Uterine cancers
"Uveal/Eye"     = "UVM",                     # Uveal melanoma
"Lymphoid"      = "DLBC",                     # Lymphoid cancer
"Leukemia"      = "LAML",                     # Leukemia
"Myeloid"       = "LAML",                     # Myeloid leukemia
"Bone Marrow"   = "LAML",                     # Bone marrow leukemia
"Body Cavity"   = "MESO",                     # Body cavity mesothelioma
"Hematologic"   = c("DLBC", "LAML"),          # Hematologic cancers
"Adrenal/SNS"   = c("ACC", "PCPG"),           # Adrenal and sympathetic nervous system
"Unknown"       = character(0),               # No TCGA mapping for unknown
"Other"         = character(0)                # No TCGA mapping for other
)
}

# Map DSMZ samples to TCGA cohorts with organ constraints; pick best- candidate
perform_organ_mapping <- function(dsmz_meta, spearman_scores, tcga_means) { # Define function
  cat("[INFO] Performing organ-constrained mapping (robust)...\n") # Print message indicating m
  organ2tcga <- get_organ_tcga_mapping() # Get organ-to-TCGA mapping
  corr_rows <- rownames(spearman_scores) # Get sample IDs from correlation matrix
  corr_cols <- colnames(spearman_scores) # Get TCGA cohort names from correlation
  tcga_codes_available <- intersect(colnames(tcga_means), corr_cols) # Find available TCGA co
  dsmz_meta %>% # Start with DSMZ metadata
    mutate( # Clean sample_id and organ
      sample_id = trimws(as.character(sample_id)), # Trim whitespace and convert to charac
      organ = ifelse(is.na(organ) | organ == "", "Other", trimws(as.character(organ))) # Repl
    ) %>%
  rowwise() %>% # Process each row individually
    mutate( # Add TCGA candidates and best match
      tcga_candidates = list({ # Create list of candidate TCGA codes
        cands0 <- organ2tcga[[organ]] # Get candidates for organ
        if (is.null(cands0)) character(0) else intersect(cands0, tcga_codes_available) # Filt
      }),
      tcga_code = { # Select best TCGA code based on correl
        sid <- sample_id # Get sample ID
        cand <- tcga_candidates # Get candidate TCGA codes
        if (!is.character(cand)) cand <- as.character(cand) # Ensure candidates are character

```

```

    if (is.na(sid) || !(sid %in% corr_rows) || length(cand) == 0L) { # Check for invalid
      NA_character_ # Return NA if invalid
    } else {
      ridx <- which(corr_rows == sid)[1] # Get row index for sample
      cand <- intersect(cand, corr_cols) # Filter candidates to available columns
      if (length(cand) == 0L) { # Check if no candidates remain
        NA_character_ # Return NA if no candidates
      } else {
        sc <- spearman_scores[ridx, cand, drop = TRUE] # Get correlation scores for candidates
        sc[!is.finite(sc)] <- -Inf # Replace non-finite scores with -Inf
        cand[which.max(sc)] # Select candidate with maximum correlation
      }
    }
  }
) %>%
ungroup() # Remove rowwise grouping
}

# Reverse mapping (TCGA project to organ)
tcga_project_to_organ_map <- function() { # Define function for TCGA-to-organ mapping
  get_organ_tcga_mapping() %>% # Get organ-to-TCGA mapping
  enframe(name = "organ", value = "codes") %>% # Convert to tibble with organ and codes
  unnest_longer(codes, values_to = "tcga_project") %>% # Expand codes to individual rows
  filter(!is.na(tcga_project)) %>% # Remove NA TCGA projects
  distinct(tcga_project, .keep_all = TRUE) %>% # Keep unique TCGA projects
  select(tcga_project, organ) # Select TCGA project and organ
}

```

```

# =====
# VISUALIZATION
# =====

# Palettes for consistent plotting
get_palettes <- function() { # Define function to return color palettes
  list( # Return list of color palettes
    okabe_ito = c("#E69F00", "#56B4E9", "#009E73", "#F0E442",
                  "#0072B2", "#D55E00", "#CC79A7", "#000000"), # Okabe-Ito colorblind-friendly
    tol_bright = c("#4477AA", "#EE6677", "#228833", "#CCBB44",
                  "#66CCEE", "#AA3377", "#BBBBBB"), # Tol bright palette
    tol_vibrant = c("#EE7733", "#0077BB", "#33BBEE", "#EE3377",
                  "#CC3311", "#009988", "#BBBBBB"), # Tol vibrant palette
    two_group = c("#E69F00", "#0072B2"), # Palette for two groups
    sequential = c("#FFFFFF", "#FF2C2C", "#FFE699", "#FFD966",
                  "#FFCC33", "#FFB300", "#E69F00", "#CC8800") # Sequential palette
  )
}

```

```

# Create violin/heatmap/bar plots + stats tables
create_plots <- function(summary_tbl, spearman_scores, dsmz_map, outdir) { # Define function
  cat("[INFO] Creating correlation plots...\n") # Print message indicating plot creation
  palettes <- get_palettes() # Retrieve color palettes

  mean_ci95 <- function(x) { # Define helper function to compute mean
    x <- x[is.finite(x)] # Remove non-finite values
    n <- length(x); m <- mean(x); se <- stats::sd(x) / sqrt(n) # Calculate n, mean, and standard error
    data.frame(y = m, ymin = m - 1.96*se, ymax = m + 1.96*se) # Return mean and CI bounds
  }

  fig_dir <- file.path(outdir, "figs") # Define figure output directory
  dir.create(fig_dir, showWarnings = FALSE, recursive = TRUE) # Create figure directory

  spearman_long <- spearman_scores %>% # Convert correlation matrix to long format
    as.data.frame() %>% # Convert to data frame
    tibble::rownames_to_column("sample_id") %>% # Add sample IDs as column
    tidyr::pivot_longer(-sample_id, names_to = "tcga_project", values_to = "rho") %>% # Pivot
    dplyr::left_join(dsmz_map %>% dplyr::select(sample_id, organ), by = "sample_id") %>% # Join
    dplyr::mutate(organ = dplyr::coalesce(organ, "Other")) # Replace NA organs with "Other"

  best_corr_data <- spearman_long %>% # Extract highest correlation per sample
    dplyr::group_by(sample_id) %>% # Group by sample ID
    dplyr::slice_max(order_by = rho, n = 1, with_ties = FALSE) %>% # Select maximum correlation
    dplyr::ungroup() # Remove grouping

  organ_summary <- best_corr_data %>% # Summarize correlations by organ
    dplyr::group_by(organ) %>% # Group by organ
    dplyr::summarise( # Compute summary statistics
      n = dplyr::n(), # Count samples
      mean = mean(rho, na.rm = TRUE), # Mean correlation
      sd = sd(rho, na.rm = TRUE), # Standard deviation
      se = sd/sqrt(n), # Standard error
      ci95_low = mean - 1.96*se, # Lower 95% CI
      ci95_high = mean + 1.96*se, # Upper 95% CI
      .groups = "drop" # Drop grouping
    ) %>%
    dplyr::arrange(dplyr::desc(mean)) # Sort by mean correlation
  readr::write_csv(organ_summary, file.path(fig_dir, "violin_rho_by_organ_summary.csv")) # Save

  kw <- rstatix::kruskal_test(best_corr_data, rho ~ organ) # Perform Kruskal-Wallis test
  dunn <- rstatix::dunn_test(best_corr_data, rho ~ organ, p.adjust.method = "BH") %>% # Perform
    rstatix::add_significance("p.adj") # Add significance symbols
  dunn_sig <- dunn %>% dplyr::filter(p.adj < 0.05) # Filter significant comparisons
  if (nrow(dunn_sig) > 0) { # Check if significant comparisons exist
    dunn_sig <- rstatix::add_xy_position(dunn_sig, x = "organ", data = best_corr_data, step.increase = 1)
  }
}

```

```

organs <- sort(unique(best_corr_data$organ))           # Get sorted unique organs
n_organs <- length(organs)                           # Count number of organs
organ_colors <- if (n_organs <= 8) palettes$okabe_ito[1:n_organs] # Use Okabe-Ito palette for
  else c(palettes$okabe_ito, palettes$tol_bright, palettes$tol_vibrant)[1:n_organs]
names(organ_colors) <- organs                        # Assign organ names to colors

kw_p <- kw$p[[1]]                                    # Extract Kruskal-Wallis p-value
subtitle_txt <- paste0("Kruskal-Wallis p = ", formatC(kw_p, format = "e", digits = 2)) # For

p1 <- ggplot(best_corr_data, aes(x = organ, y = rho)) + # Initialize violin plot
  geom_violin(aes(fill = organ), trim = FALSE, alpha = 0.6, color = "black", size = 0.5) +
  geom_jitter(aes(color = organ, shape = organ), width = 0.15, alpha = 0.7, size = 1.5) +
  stat_summary(fun.data = mean_ci95, geom = "pointrange", color = "black", size = 0.8, fatten = 1) +
  scale_fill_manual(values = organ_colors, guide = "none") + # Set fill colors, hide legend
  scale_color_manual(values = organ_colors, guide = "none") + # Set point colors, hide legend
  scale_shape_manual(values = rep(c(16, 17, 15, 18, 8, 4, 3, 7), length.out = n_organs), guide = "none") +
  theme_bw(base_size = 12) +                          # Use black-and-white theme
  theme(                                                # Customize theme
    axis.text.x = element_text(angle = 45, hjust = 1, size = 11), # Rotate x-axis labels
    axis.text.y = element_text(size = 11),              # Set y-axis text size
    axis.title = element_text(size = 12, face = "bold"), # Bold axis titles
    plot.title = element_text(size = 14, face = "bold"), # Bold plot title
    plot.subtitle = element_text(size = 11),            # Set subtitle size
    panel.grid.minor = element_blank(),                # Remove minor grid lines
    panel.grid.major.x = element_blank(),              # Remove major x grid lines
  ) +
  labs(                                                # Add plot labels
    x = "Organ/Tissue Type",                          # X-axis label
    y = "Best Spearman rho",                          # Y-axis label
    title = "Distribution of Best DSMZ-TCGA Correlations by Organ/Tissue", # Plot title
    subtitle = subtitle_txt                          # Plot subtitle
  )
if (nrow(dunn_sig) > 0) {                             # Check if significant comparisons exist
  p1 <- p1 + ggpubr::stat_pvalue_manual(dunn_sig, label = "p.adj.signif", hide.ns = TRUE, tip = FALSE)
}
print(p1)                                             # Print violin plot
ggsave(file.path(fig_dir, "violin_rho_by_organ.pdf"), p1, width = 12, height = 7) # Save violin plot

# Heatmap of top 25
topN <- head(dplyr::arrange(summary_tbl, dplyr::desc(best_rho_overall))$sample_id, 25) # Select top 25 samples
hm <- spearman_scores[topN, , drop = FALSE]          # Subset correlation matrix for top 25
colors_div <- rev(RColorBrewer::brewer.pal(11, "RdYlBu")) # Create reversed RdYlBu palette
hp <- pheatmap::pheatmap(                            # Create heatmap
  hm, cluster_rows = TRUE, cluster_cols = TRUE,      # Enable clustering
  color = colorRampPalette(colors_div)(100),         # Use smooth color gradient
  main = "Top 25 DSMZ Samples: Spearman Correlation with TCGA Cohorts", # Heatmap title
  fontsize = 9, fontsize_row = 7, fontsize_col = 8, # Set font sizes

```

```

border_color = "grey60", silent = TRUE # Set border color, suppress output
)
grid::grid.newpage(); grid::grid.draw(hp$gtable) # Render heatmap
pdf(file.path(fig_dir, "heatmap_top25.pdf"), width = 12, height = 10) # Open PDF device
grid::grid.newpage(); grid::grid.draw(hp$gtable) # Draw heatmap in PDF
dev.off() # Close PDF device

# Stacked bar: DSMZ organ vs assigned TCGA cohort
assignment_data <- dsmz_map %>% # Prepare data for bar chart
  dplyr::mutate(tcga_assignment = ifelse(is.na(tcga_code) | tcga_code == "", "Unassigned", tcga_code))
  dplyr::count(organ, tcga_assignment, name = "count") # Count samples by organ and TCGA assignment
totals <- assignment_data %>% dplyr::group_by(organ) %>% dplyr::summarise(total = sum(count))
organ_order <- totals$organ # Define organ order
assignment_data$organ <- factor(assignment_data$organ, levels = organ_order) # Convert organ to factor
totals$organ <- factor(totals$organ, levels = organ_order) # Convert totals organ to factor
assignment_labels <- assignment_data %>% # Calculate label positions
  dplyr::group_by(organ) %>% # Group by organ
  dplyr::arrange(organ, tcga_assignment, .by_group = TRUE) %>% # Sort within groups
  dplyr::mutate(pos = cumsum(count) - count/2) %>% # Calculate midpoint for labels
  dplyr::ungroup() # Remove grouping
tcga_assignments <- sort(unique(assignment_data$tcga_assignment)) # Get unique TCGA assignments
n_tcga <- length(tcga_assignments) # Count TCGA assignments
tcga_colors <- c(palettes$okabe_ito, palettes$tol_bright, palettes$tol_vibrant)[1:n_tcga] #
names(tcga_colors) <- tcga_assignments # Name colors by assignments
label_df <- dplyr::filter(assignment_labels, count >= 2) # Filter labels for segments with
assignment_data <- assignment_data %>% dplyr::mutate(tcga_assignment = factor(tcga_assignment, levels = tcga_assignments))
label_df <- label_df %>% dplyr::mutate(tcga_assignment = factor(tcga_assignment, levels = tcga_assignments))

p3 <- ggplot(assignment_data, aes(x = organ, y = count, fill = tcga_assignment)) + # Initial plot
  geom_col(color = "grey15", linewidth = 0.2) + # Add bars with grey outline
  geom_text(data = label_df, inherit.aes = FALSE, aes(x = organ, y = pos, label = count), size = 8) +
  geom_text(data = totals, inherit.aes = FALSE, aes(x = organ, y = total, label = total), vjust = "bottom") +
  scale_fill_manual(values = tcga_colors, name = "TCGA Assignment") + # Set fill colors
  scale_y_continuous(expand = expansion(mult = c(0.02, 0.12))) + # Adjust y-axis spacing
  theme_bw(base_size = 11) + # Use black-and-white theme
  labs(x = "Organ/Tissue Type", y = "Number of Cell Lines", title = "DSMZ Cell Line Assignment") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1), legend.position = "bottom") + #
  guides(fill = guide_legend(nrow = 2, byrow = TRUE)) # Use 2-row legend
print(p3) # Print bar chart
ggsave(file.path(fig_dir, "organ_tcga_assignment.pdf"), p3, width = 14, height = 9) # Save plot

readr::write_csv(kw, file.path(fig_dir, "violin_kw_results.csv")) # Save Kruskal-Wallis results
readr::write_csv(dunn, file.path(fig_dir, "violin_dunn_results.csv")) # Save Dunn test results
cat("[INFO] Plots saved to: ", fig_dir, "\n") # Print message indicating plot save location
}

```

```

# =====
# PCA + BATCH EFFECT ASSESSMENT
# =====

# PCA + R2 of PCs ~ dataset indicator
pc_dataset_s_ <- function(M_log, dataset_factor, top_n = 10, outdir = NULL, title_tag = "dataset") {
  if (!is.matrix(M_log) && !is.data.frame(M_log)) stop("M_log must be a matrix or data frame")
  if (ncol(M_log) != length(dataset_factor)) stop("Length mismatch between M_log columns and dataset_factor")
  palettes <- get_palettes() # Retrieve color palettes

  pc <- prcomp(t(M_log), scale. = TRUE) # Perform PCA on transposed log-transformed data
  var_explained <- (pc$sdev^2) / sum(pc$sdev^2) # Calculate proportion of variance explained

  if (is.null(top_n) || top_n > ncol(pc$x)) { # Check if top_n is invalid
    cum_var <- cumsum(var_explained) # Calculate cumulative variance
    top_n <- which.max(cum_var >= 0.9) # Select PCs explaining 90% variance
    top_n <- min(top_n, ncol(pc$x)) # Limit to available PCs
  }

  get_r2 <- function(y) unname(summary(lm(y ~ dataset_factor))$r.squared) # Define function to get R2
  pcs_to_check <- seq_len(min(top_n, ncol(pc$x))) # Define PCs to analyze
  r2_values <- sapply(pcs_to_check, function(i) get_r2(pc$x[, i])) # Compute R2 for each PC

  df_r2 <- data.frame( # Create R2 summary table
    PC = paste0("PC", pcs_to_check), # PC names
    R2 = r2_values, # R2 values
    VariancePct = var_explained[pcs_to_check] * 100 # Variance percentage
  )
  df_r2$PC <- factor(df_r2$PC, levels = df_r2$PC) # Convert PC to factor for plotting

  p1 <- ggplot(df_r2, aes(x = PC, y = R2, fill = VariancePct)) + # Initialize R2 bar plot
    geom_col(color = "black", size = 0.3) + # Add bars with black outline
    scale_fill_viridis_c(option = "plasma", name = "Variance explained (%)", # Use viridis color palette
      limits = c(0, max(var_explained) * 100)) +
    coord_cartesian(ylim = c(0, 1)) + # Fix y-axis range
    theme_bw(base_size = 12) + # Use black-and-white theme
    theme(axis.text.x = element_text(angle = 45, hjust = 1), # Rotate x-axis labels
      axis.title = element_text(face = "bold"), # Bold axis titles
      plot.title = element_text(face = "bold"), # Bold plot title
      legend.position = "right") + # Place legend on right
    labs(title = "Variance in PCs Explained by Dataset Factor", # Add labels
      subtitle = sprintf("Top %d PCs (%.1f%% cumulative variance)", top_n, sum(var_explained[1:top_n])),
      x = "Principal Component",
      y = expression(R^2 ~ "(dataset factor)"))

  pc_df <- as.data.frame(pc$x[, 1:2]) # Create data frame with PC1 and PC2
  pc_df$Group <- dataset_factor # Add dataset factor

```

```

pc1_var <- round(var_explained[1] * 100, 1)      # Calculate PC1 variance percentage
pc2_var <- round(var_explained[2] * 100, 1)      # Calculate PC2 variance percentage
group_levels <- unique(dataset_factor)          # Get unique dataset groups
group_colors <- setNames(get_palettes())$two_group[seq_along(group_levels)], group_levels) #

p2 <- ggplot(pc_df, aes(x = PC1, y = PC2, color = Group, shape = Group)) + # Initialize PCA
  geom_point(size = 2.5, alpha = 0.8, stroke = 0.5) + # Add points
  stat_ellipse(aes(fill = Group), geom = "polygon", alpha = 0.15, linetype = "dashed", size = 1) +
  scale_color_manual(values = group_colors, name = title_tag) + # Set point colors
  scale_fill_manual(values = group_colors, guide = "none") + # Set ellipse fill, hide legend
  scale_shape_manual(values = c(16, 17)[seq_along(group_levels)], name = title_tag) + # Set
  theme_bw(base_size = 12) + # Use black-and-white theme
  theme(legend.position = "right", # Place legend on right
        axis.title = element_text(face = "bold"), # Bold axis titles
        plot.title = element_text(face = "bold"), # Bold plot title
        panel.grid.minor = element_blank()) + # Remove minor grid lines
  labs(title = paste("PCA: PC1 vs PC2 -", title_tag, "Comparison"), # Add labels
        x = paste0("PC1 (", pc1_var, "% variance)"),
        y = paste0("PC2 (", pc2_var, "% variance)")) +
  coord_fixed(ratio = 1) # Set equal aspect ratio

combined_plot <- p1 / p2 + patchwork::plot_layout(heights = c(1, 1.2)) # Combine plots vertically
print(combined_plot) # Print combined plot

if (!is.null(outdir)) { # Check if output directory is specified
  fig_dir <- file.path(outdir, "figs") # Define figure directory
  dir.create(fig_dir, showWarnings = FALSE, recursive = TRUE) # Create figure directory
  ggsave(file.path(fig_dir, paste0("PCA_s_", title_tag, ".pdf")), combined_plot, width = 10,
  write.csv(df_r2, file.path(fig_dir, paste0("PCA_s_", title_tag, "_R2_table.csv")), row.names = NULL)
}
return(list(r2_table = df_r2, pc = pc, plot = combined_plot, # Return PCA results
  summary = list(total_variance = sum(var_explained[pcs_to_check]),
    significant_pcs = sum(r2_values > 0.1))))
}

# Scree plot
make_scree_plot <- function(pc, outdir, title_tag, k = 20) { # Define function to create scree plot
  palettes <- get_palettes() # Retrieve color palettes
  var_expl <- (pc$sdev^2) / sum(pc$sdev^2) # Calculate variance explained
  k <- min(k, length(var_expl)) # Limit k to available PCs
  df <- data.frame( # Create data frame for plotting
    PC = factor(paste0("PC", seq_len(k)), levels = paste0("PC", seq_len(k))), # PC names
    Var = var_expl[seq_len(k)] * 100, # Variance percentage
    Cum = cumsum(var_expl[seq_len(k)]) * 100 # Cumulative variance percentage
  )
  p <- ggplot(df, aes(PC, Var)) + # Initialize scree plot
  geom_col(fill = palettes$okabe_ito[5], color = "black", size = 0.4, alpha = 0.85) + # Add

```

```

geom_line(aes(y = Cum, group = 1), color = palettes$okabe_ito[6], size = 1.5) + # Add cumulative variance line
geom_point(aes(y = Cum), color = palettes$okabe_ito[6], fill = "white", shape = 21, size = 10) + # Add cumulative variance points
theme_bw(base_size = 12) + # Use black-and-white theme
theme(axis.text.x = element_text(angle = 45, hjust = 1), # Rotate x-axis labels
      axis.title = element_text(face = "bold"), # Bold axis titles
      plot.title = element_text(face = "bold"), # Bold plot title
      plot.caption = element_text(hjust = 0, face = "italic"), # Italic caption
      panel.grid.minor = element_blank()) + # Remove minor grid lines
labs(title = paste("Scree Plot -", title_tag), # Add labels
     x = "Principal Component",
     y = "Variance Explained (%)",
     caption = "Blue bars: per-PC variance; Orange line/points: cumulative variance")
if (!is.null(outdir)) { # Check if output directory is specified
  dir.create(file.path(outdir, "figs"), showWarnings = FALSE, recursive = TRUE) # Create figure directory
  ggsave(file.path(outdir, "figs", paste0("scree_", title_tag, ".pdf")), p, width = 10, height = 10)
}
return(p) # Return scree plot
}

# PC + covariate R² heatmap
pc_covariate_r2_matrix_ <- function(pc, sample_df, covariates, n_pcs = 10, outdir = NULL, tag = "PC_covariate_R2_matrix_") {
  stopifnot(is.list(pc), !is.null(pc$x)) # Validate PCA object
  stopifnot("sample" %in% colnames(sample_df)) # Validate sample column in metadata
  rn <- rownames(pc$x) # Get PCA sample IDs
  if (is.null(rn)) stop("pc$x must have rownames (sample IDs).") # Stop if no sample IDs
  md <- sample_df[match(rn, sample_df$sample), , drop = FALSE] # Align metadata to PCA sample IDs
  k <- min(n_pcs, ncol(pc$x)); pcs <- paste0("PC", seq_len(k)) # Define PCs to analyze
  Y <- pc$x[, seq_len(k), drop = FALSE] # Extract PC scores
  covariates <- covariates[covariates %in% colnames(md)] # Filter valid covariates
  if (!length(covariates)) stop("No valid covariates found in metadata.") # Stop if no valid covariates
  r2_mat <- matrix(NA_real_, nrow = k, ncol = length(covariates), dimnames = list(pcs, covariates))
  for (j in seq_along(covariates)) { # Loop through covariates
    v <- md[[covariates[j]]] # Extract covariate values
    if (is.character(v) || is.logical(v)) v <- factor(v) # Convert to factor if needed
    for (i in seq_len(k)) { # Loop through PCs
      y <- Y[, i]; ok <- is.finite(y) & !is.na(v) # Identify valid observations
      if (!any(ok)) next # Skip if no valid observations
      if (is.factor(v) && nlevels(droplevels(v[ok])) < 2) next # Skip if factor has <2 levels
      fit <- try(lm(y[ok] ~ v[ok]), silent = TRUE) # Fit linear model
      if (!inherits(fit, "try-error")) r2_mat[i, j] <- summary(fit)$r.squared # Store R²
    }
  }
}

if (!is.null(outdir)) { # Check if output directory is specified
  dir.create(outdir, showWarnings = FALSE, recursive = TRUE) # Create output directory
  write.csv(as.data.frame(r2_mat), file.path(outdir, paste0("PC_covariate_R2_matrix_", tag, ".csv")), as.is = TRUE)
}
r2_df <- as.data.frame(r2_mat) %>% tibble::rownames_to_column("PC") %>% # Convert R² matrix to data frame

```

```

tidyr::pivot_longer(-PC, names_to = "Covariate", values_to = "R2")
p <- ggplot(r2_df, aes(Covariate, PC, fill = R2)) + # Initialize heatmap
  geom_tile(color = "white", size = 0.5) +        # Add tiles
  scale_fill_viridis_c(option = "plasma", na.value = "grey95", limits = c(0, 1), name = "R2")
  theme_bw(base_size = 11) +                      # Use black-and-white theme
  theme(axis.text.x = element_text(angle = 45, hjust = 1, size = 10), # Customize theme
        axis.text.y = element_text(size = 10),
        axis.title = element_text(face = "bold", size = 11),
        plot.title = element_text(face = "bold", size = 12),
        panel.grid = element_blank(),
        legend.position = "right") +
  labs(title = paste0("PC vs covariate R2 ("), tag, ")"), # Add labels
        x = "Covariate", y = "Principal Component")
if (isTRUE(show_plot)) print(p)                  # Print plot if requested
if (!is.null(outdir)) ggsave(file.path(outdir, paste0("PC_covariate_R2_matrix_", tag, ".pdf")), p)
invisible(list(r2 = r2_mat, plot = p))           # Return R2 matrix and plot
}

```

```

# =====
# OUTPUT HELPERS
# =====

save_results <- function(spearman_scores, summary_tbl, dsmz_map, outdir) { # Define function
  cat("[INFO] Saving correlation tables...\n") # Print message indicating saving
  all_wide <- as.data.frame(spearman_scores) %>% tibble::rownames_to_column("sample_id") # Convert to wide
  all_long <- all_wide %>% pivot_longer(-sample_id, names_to = "tcga_cohort", values_to = "rho") # Convert to long
  write.csv(all_wide, file.path(outdir, "spearman_all_wide.csv"), row.names = FALSE) # Save wide
  write.csv(all_long, file.path(outdir, "spearman_all_long.csv"), row.names = FALSE) # Save long
  saveRDS(spearman_scores, file.path(outdir, "spearman_all.rds")) # Save correlation matrix as RDS
  write.csv(summary_tbl, file.path(outdir, "spearman_best_by_sample.csv"), row.names = FALSE)
  write.csv(dsmz_map %>% select(sample_id, organ, tcga_code), file.path(outdir, "dsmz_tcga_map.csv"), row.names = FALSE)
}

save_debug_info <- function(tcga_counts, dsmz_raw, common_genes, outdir) { # Define function
  writeLines(utils::capture.output(head(rownames(tcga_counts))), file.path(outdir, "head_tcga_counts.txt"))
  writeLines(utils::capture.output(head(dsmz_raw$Ensembl_ID)), file.path(outdir, "head_dsmz_raw.txt"))
  writeLines(common_genes[1:min(100, length(common_genes))], file.path(outdir, "common_genes.txt"))
  sink(file.path(outdir, "sessionInfo.txt")); print(sessionInfo()); sink() # Save session info
}

```

```

# =====
# MAIN PIPELINE
# =====

main <- function() { # Define main pipeline function
  cat("[INFO] Starting pipeline...\n") # Print message indicating pipeline start

  # Load input datasets

```

```

tcga <- load_tcga_data(config$tcga_se_rds)           # Load TCGA data
dsmz <- load_dsmz_data(config$dsmz_rds, config$dsmz_meta_csv) # Load DSMZ data

# Harmonize & build matrices
tcga_counts <- harmonize_gene_ids(tcga$counts); storage.mode(tcga_counts) <- "double" # Harmonize
dsmz_counts <- build_dsmz_matrix(dsmz$raw); storage.mode(dsmz_counts) <- "double" # Build DSMZ matrices

# Align DSMZ metadata
al <- align_metadata(dsmz$meta, dsmz_counts, config$outdir) # Align metadata with counts
dsmz_meta <- al$meta # Extract aligned metadata
dsmz_counts <- al$counts # Extract aligned counts
stopifnot("organ" %in% colnames(dsmz_meta)) # Verify organ column exists

# Common genes
common <- intersect(rownames(tcga_counts), rownames(dsmz_counts)) # Find common genes
cat(sprintf("[INFO] Shared genes: %d\n", length(common))) # Print number of shared genes
if (length(common) < config$min_shared) cat("[WARN] Few shared genes; check Ensembl IDs\n")
tcga_counts <- tcga_counts[common, , drop=FALSE] # Subset TCGA counts to common genes
dsmz_counts <- dsmz_counts[common, , drop=FALSE] # Subset DSMZ counts to common genes

# ----- Build merged matrices & define batches -----
Xc_raw <- cbind(tcga_counts, dsmz_counts) # Combine TCGA and DSMZ raw counts
batch <- factor(c(rep("TCGA", ncol(tcga_counts)), rep("DSMZ", ncol(dsmz_counts)))) # Define batches

# ----- PCA BEFORE correction (baseline) -----
M0_log <- logCPM(Xc_raw) # Compute log2-CPM for uncorrected data

# ----- Batch correction (ONE place) -----
cat("[INFO] Batch adjustment (global)... \n") # Print message indicating batch correction
method <- tolower(config$batch_adjust_method) # Get batch correction method

if (method == "none") { # Check if no batch correction
  cat("[INFO] Skipping batch correction. \n") # Print message
  M1_log <- M0_log # Use uncorrected data
} else if (method == "combat_seq") { # Check if ComBat-seq is selected
  if (min(table(batch)) < 2) { # Check for singleton batches
    cat("[WARN] Singleton batch detected; falling back to ComBat on logCPM. \n") # Warn about singletons
    M1_log <- sva::ComBat(dat = as.matrix(M0_log), batch = batch, # Apply standard ComBat
                        mod = NULL, par.prior = TRUE, prior.plots = FALSE, mean.only = FALSE)
  } else {
    Xc <- Xc_raw # Use raw counts
    Xc[is.na(Xc)] <- 0 # Replace NA with 0
    Xc <- round(Xc) # Round to integers
    set.seed(42) # Set seed for reproducibility
    Xc_adj <- sva::ComBat_seq(as.matrix(Xc), batch = batch) # Apply ComBat-seq
    M1_log <- logCPM(Xc_adj) # Log-transform corrected counts
  }
}

```

```

} else if (method == "combat") { # Check if standard ComBat is selected
  M1_log <- sva::ComBat(dat = as.matrix(M0_log), batch = batch, # Apply standard ComBat
    mod = NULL, par.prior = TRUE, prior.plots = FALSE, mean.only = FALSE)
} else {
  stop("[ERROR] Unknown batch_adjust_method. Use 'none', 'combat_seq', or 'combat'.") # Stop
}

# ----- Optional purity adjustment on TCGA (log scale) -----
purity <- load_purity_data(tcga$se, config$purity_tsv) # Load purity data
if (!is.null(purity)) { # Check if purity data exists
  tcga_cols <- colnames(tcga_counts) # Get TCGA sample columns
  tcga_log_adj <- purity_adjust_log(M1_log[, tcga_cols, drop=FALSE], purity) # Adjust TCGA
  M1_log[, tcga_cols] <- tcga_log_adj # Update adjusted TCGA data
}

# ----- Correlation analysis (on adjusted log2-CPM) -----
tcga_cols <- colnames(tcga_counts) # Get TCGA sample columns
dsmz_cols <- colnames(dsmz_counts) # Get DSMZ sample columns
tcga_log_mat <- M1_log[, tcga_cols, drop=FALSE] # Subset TCGA log matrix
dsmz_log_mat <- M1_log[, dsmz_cols, drop=FALSE] # Subset DSMZ log matrix
tcga_res <- compute_tcga_means(tcga$se, tcga_log_mat) # Compute TCGA cohort means
tcga_means <- tcga_res$means # Extract TCGA means
stopifnot(identical(rownames(tcga_means), rownames(dsmz_log_mat))) # Verify gene alignment
sel_genes_corr <- select_variable_genes(tcga_means, config$var_gene_set) # Select variable
spearman <- compute_correlations(dsmz_log_mat, tcga_means, dsmz_meta, sel_genes_corr) # Compute
dsmz_map <- perform_organ_mapping(dsmz_meta, spearman, tcga_means) # Map DSMZ samples to TCGA
best_idx <- max.col(spearman, ties.method="first") # Find index of maximum correlation
best_overall <- tibble( # Create summary of best correlations
  sample_id = rownames(spearman), # Sample IDs
  best_tcga_overall = colnames(spearman)[best_idx], # Best TCGA cohort
  best_rho_overall = spearman[cbind(seq_len(nrow(spearman)), best_idx)] # Best correlation
)
rho_allowed <- rep(NA_real_, nrow(dsmz_map)) # Initialize vector for organ-constrained
ok <- !is.na(dsmz_map$tcga_code) # Identify valid TCGA assignments
if (any(ok)) { # Check if valid assignments exist
  for (i in which(ok)) { # Loop through valid assignments
    sample_id <- dsmz_map$sample_id[i]; tcga_code <- dsmz_map$tcga_code[i] # Get sample and
    rho_allowed[i] <- spearman[sample_id, tcga_code] # Extract correlation
  }
}

summary_tbl <- dsmz_map %>% select(sample_id, organ, tcga_code) %>% # Create summary table
  left_join(best_overall, by="sample_id") %>% mutate(rho_allowed = rho_allowed) # Join and

create_plots(summary_tbl, spearman, dsmz_map, config$outdir) # Generate plots
save_results(spearman, summary_tbl, dsmz_map, config$outdir) # Save results
save_debug_info(tcga_counts, dsmz$raw, common, config$outdir) # Save debug info

```

```

# ----- PCA before/after (log scale consistently) -----
tcga_lab <- sub("^TCGA-", "", as.character(tcga_res$labels[colnames(tcga_log_mat)])) # Clean
tcga_df <- tibble(sample = colnames(tcga_log_mat), dataset = "TCGA", tcga_project = tcga_lab)
proj_map <- tcga_project_to_organ_map() # Get TCGA-to-organ mapping
tcga_df <- tcga_df %>% left_join(proj_map, by = "tcga_project") %>% mutate(organ = coalesce(
dsmz_df <- tibble(sample = colnames(dsmz_log_mat), dataset = "DSMZ", tcga_project = NA_character_)
  left_join(dsmz_meta %>% select(sample_id, organ), by = c("sample" = "sample_id")) %>%
  mutate(organ = coalesce(organ, "Other")) # Replace NA organs
sample_df <- bind_rows(tcga_df, dsmz_df) # Combine metadata
sel_genes_pca <- select_variable_genes(M0_log, config$var_gene_set) # Select genes for PCA
M0_log_pca <- M0_log[sel_genes_pca, , drop=FALSE] # Subset uncorrected data
M1_log_pca <- M1_log[sel_genes_pca, , drop=FALSE] # Subset corrected data

cat("[INFO] PCA BEFORE batch adjustment...\n") # Print message for pre-correction PCA
r2_before <- pc_dataset_s(M0_log_pca, factor(sample_df$dataset), top_n = 10,outdir = config$outdir)
scree_before <- make_scree_plot(r2_before$pc, config$outdir, "before") # Create scree plot
r2_cov_before <- pc_covariate_r2_matrix_(pc = r2_before$pc, sample_df = sample_df, # Compute
  covariates = c("dataset", "organ", "tcga_project"),
  n_pcs = 10, outdir = config$outdir, tag = "before", sl

cat("[INFO] PCA AFTER batch adjustment...\n") # Print message for post-correction PCA
r2_after <- pc_dataset_s(M1_log_pca, factor(sample_df$dataset), top_n = 10, outdir = config$outdir)
scree_after <- make_scree_plot(r2_after$pc, config$outdir, "after") # Create scree plot
r2_cov_after <- pc_covariate_r2_matrix_(pc = r2_after$pc, sample_df = sample_df, # Compute
  covariates = c("dataset", "organ", "tcga_project"),
  n_pcs = 10, outdir = config$outdir, tag = "after", sl

cat("\n**PC variance explained by dataset (R^2):**\n\n") # Print header for R^2 comparison
r2_comparison <- tibble( # Create R^2 comparison table
  Stage = c("Before Batch Correction", "After Batch Correction"), # Stages
  PC1_R2 = c(r2_before$r2_table$R2[r2_before$r2_table$PC == "PC1"], # PC1 R^2
    r2_after$r2_table$R2[r2_after$r2_table$PC == "PC1"]),
  PC2_R2 = c(r2_before$r2_table$R2[r2_before$r2_table$PC == "PC2"], # PC2 R^2
    r2_after$r2_table$R2[r2_after$r2_table$PC == "PC2"])
)
print(r2_comparison) # Print R^2 comparison

cat(sprintf("[INFO] Pipeline completed successfully. Results saved to: %s\n", config$outdir))
list(spearman_scores = spearman, # Return key results
  summary_table = summary_tbl,
  dsmz_mapping = dsmz_map,
  r2_comparison = r2_comparison)
}

```

```

# =====
# EXECUTION
# =====

```

```

if (!interactive()) {                                # Check if running in batch mode
  results <- main()                                   # Run pipeline in batch mode
} else {
  cat("[INFO] Running in interactive mode. Call main() to execute pipeline.\n") # Print messa
}

```

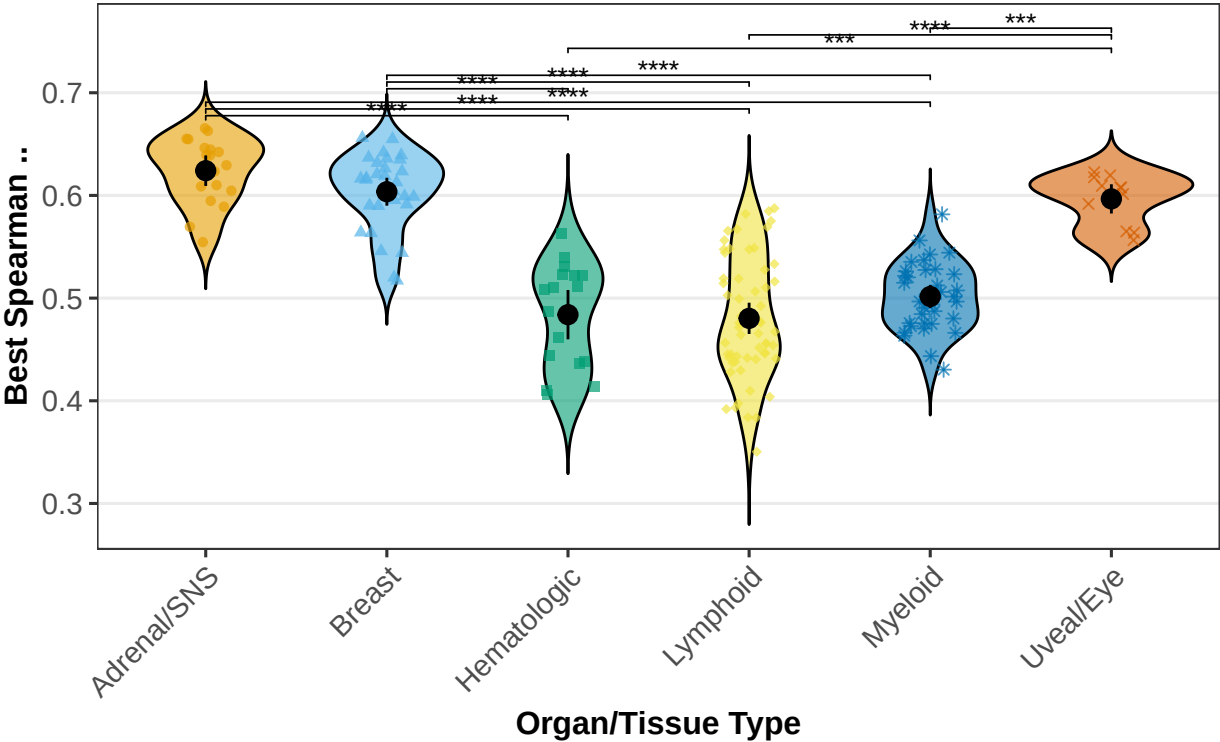
```

## [INFO] Starting pipeline...
## [INFO] Loading TCGA data...
## [INFO] TCGA dims: 60660 genes x 10272 samples
## [INFO] Loading DSMZ data...
## [INFO] DSMZ table: 60660 rows x 170 cols
## [INFO] Harmonizing TCGA gene IDs...
## [INFO] Building DSMZ count matrix...
## [INFO] Aligning metadata to counts...
## [INFO] Matched: 167 | Unmatched(meta): 0 | Unmatched(counts): 0
## [INFO] Shared genes: 60616
## [INFO] Batch adjustment (global)...
## Found 2 batches
## Using null model in ComBat-seq.
## Adjusting for 0 covariate(s) or covariate level(s)
## Estimating dispersions
## Fitting the GLM model
## Shrinkage off - using GLM estimates for parameters
## Adjusting the data
## [INFO] Using purity from SummarizedExperiment
## [INFO] Applying purity adjustment on log2-CPM...
## [INFO] Removing 878 purity-associated genes
## [INFO] Variable-gene setting: tcga_5k (5000 genes used)
## [INFO] Computing Spearman correlations...
## [INFO] Performing organ-constrained mapping (robust)...
## [INFO] Creating correlation plots...

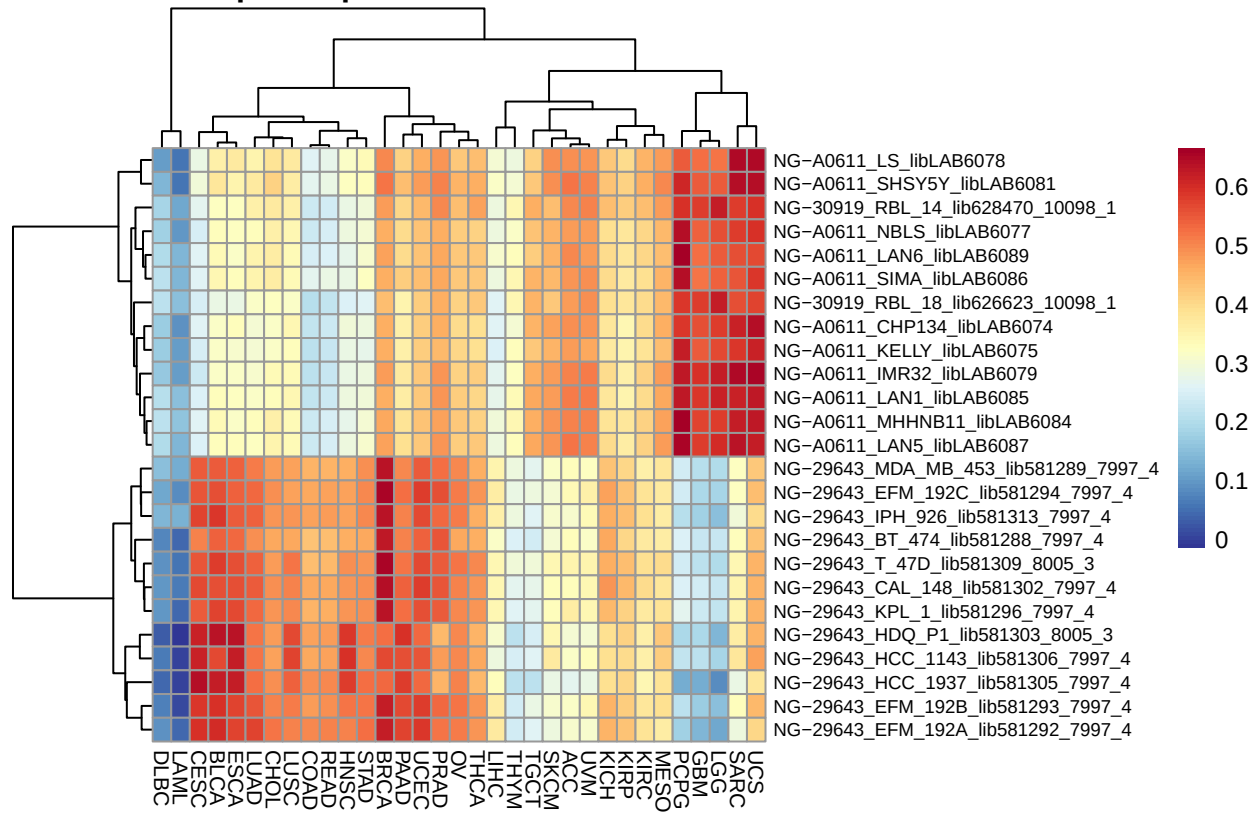
```

Distribution of Best DSMZ...TCGA Correlations by Organ/Tissue

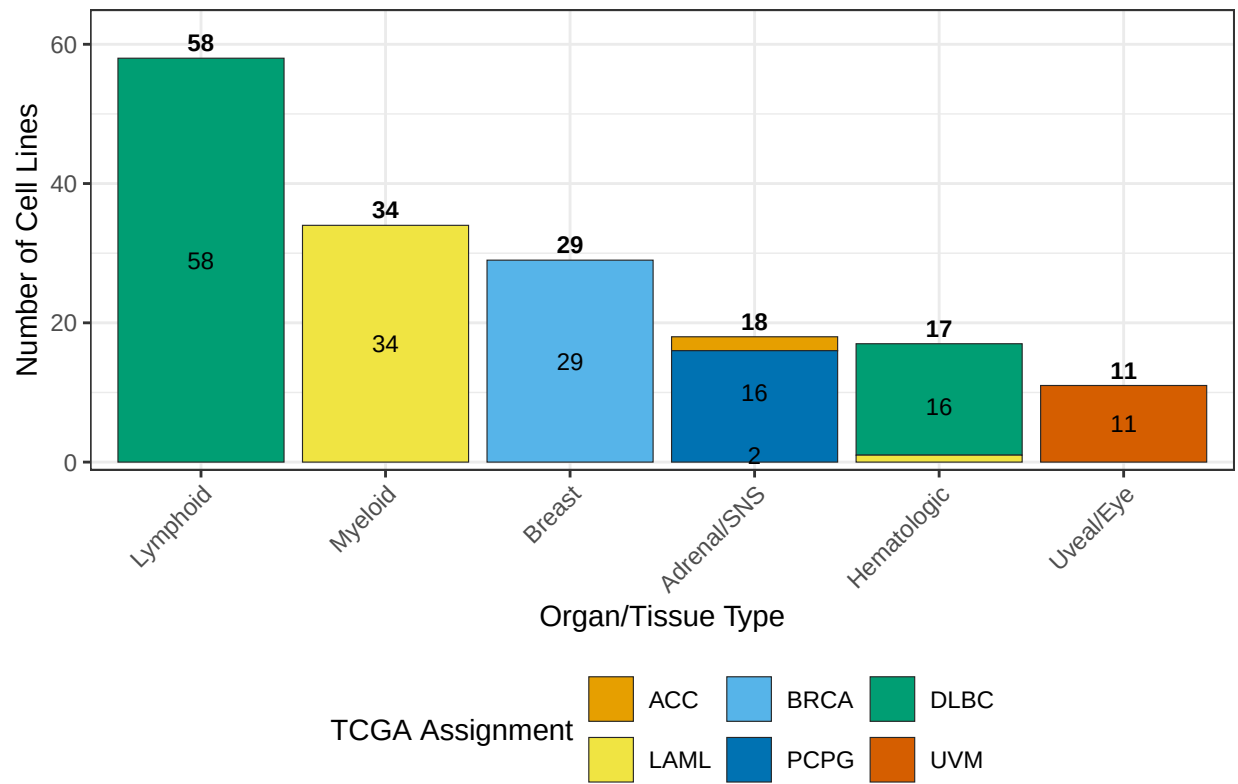
Kruskal...Wallis p = 5.94e-21



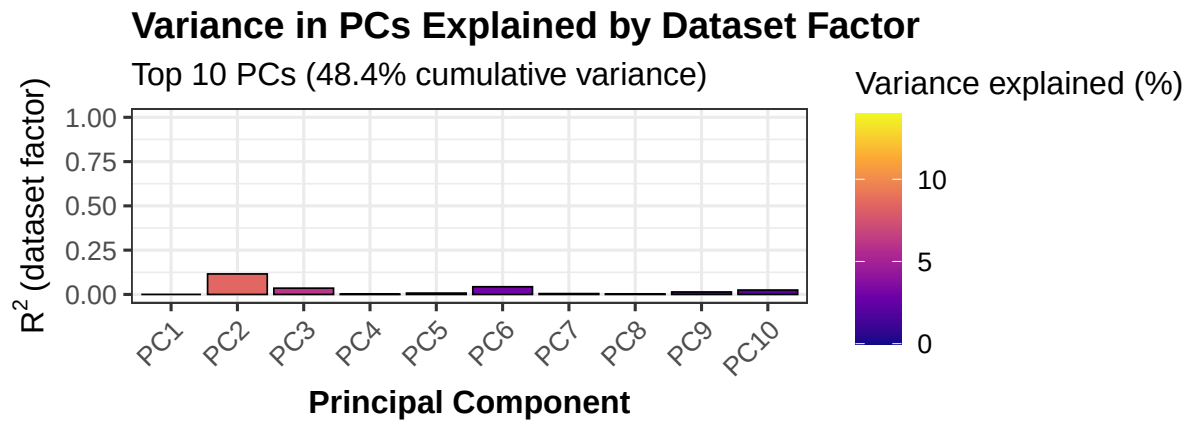
## 25 DSMZ Samples: Spearman Correlation with TCGA Cohorts



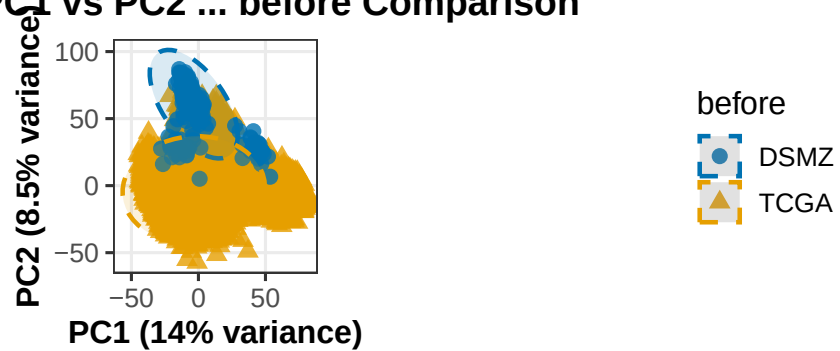
DSMZ Cell Line Assignments to TCGA Projects by Organ/Tissue

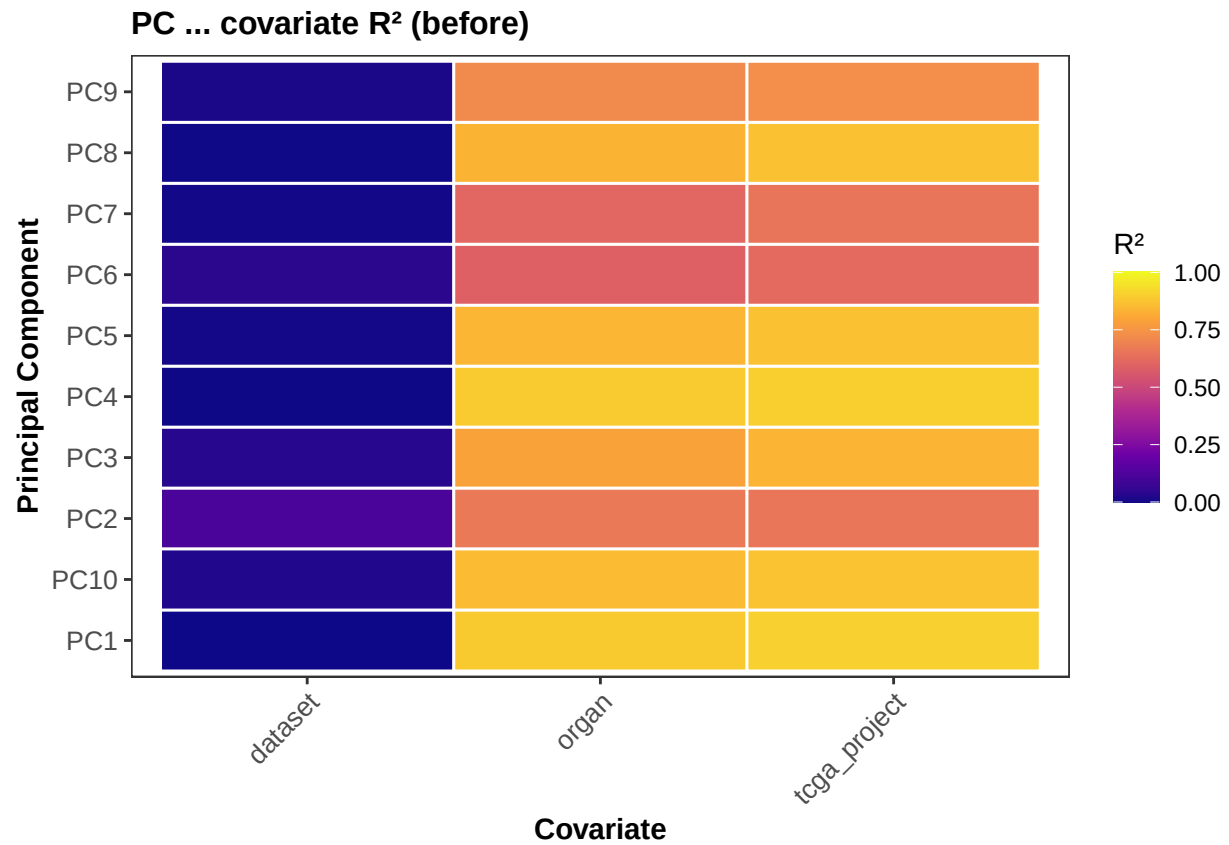


```
## [INFO] Plots saved to: /home/chu25/dsmz/results/correlation4/figs
## [INFO] Saving correlation tables...
## [INFO] Variable-gene setting: tcga_5k (5000 genes used)
## [INFO] PCA BEFORE batch adjustment...
```

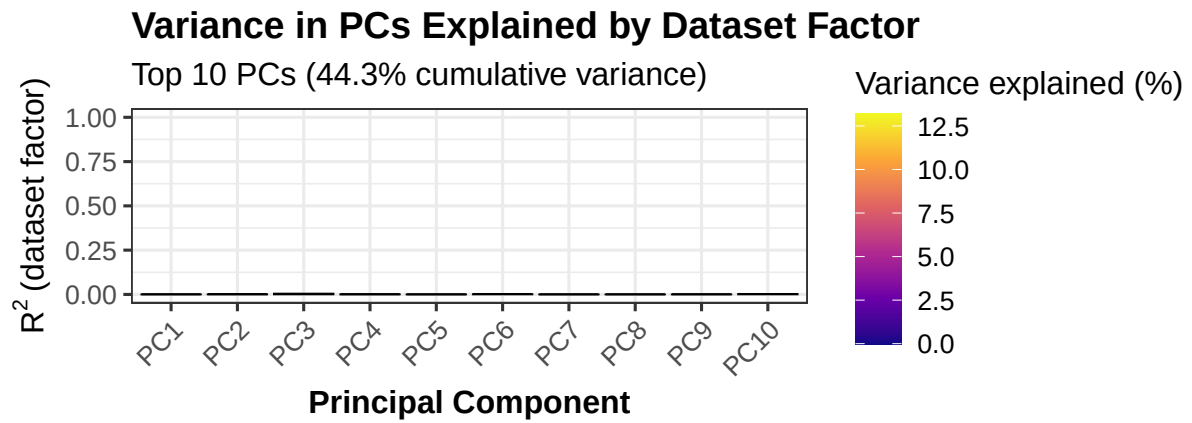


### PCA: PC1 vs PC2 ... before Comparison

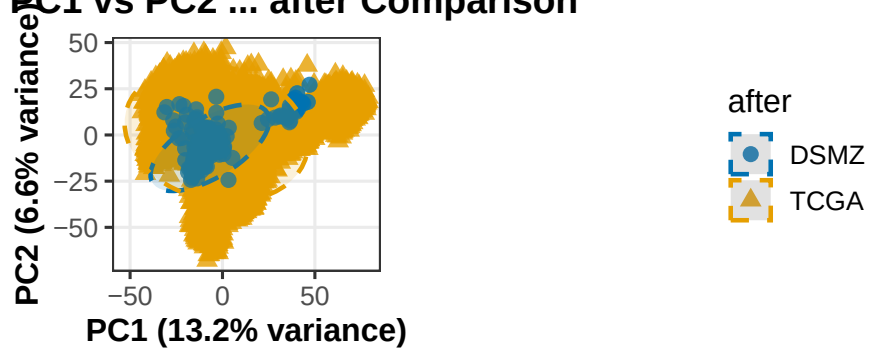


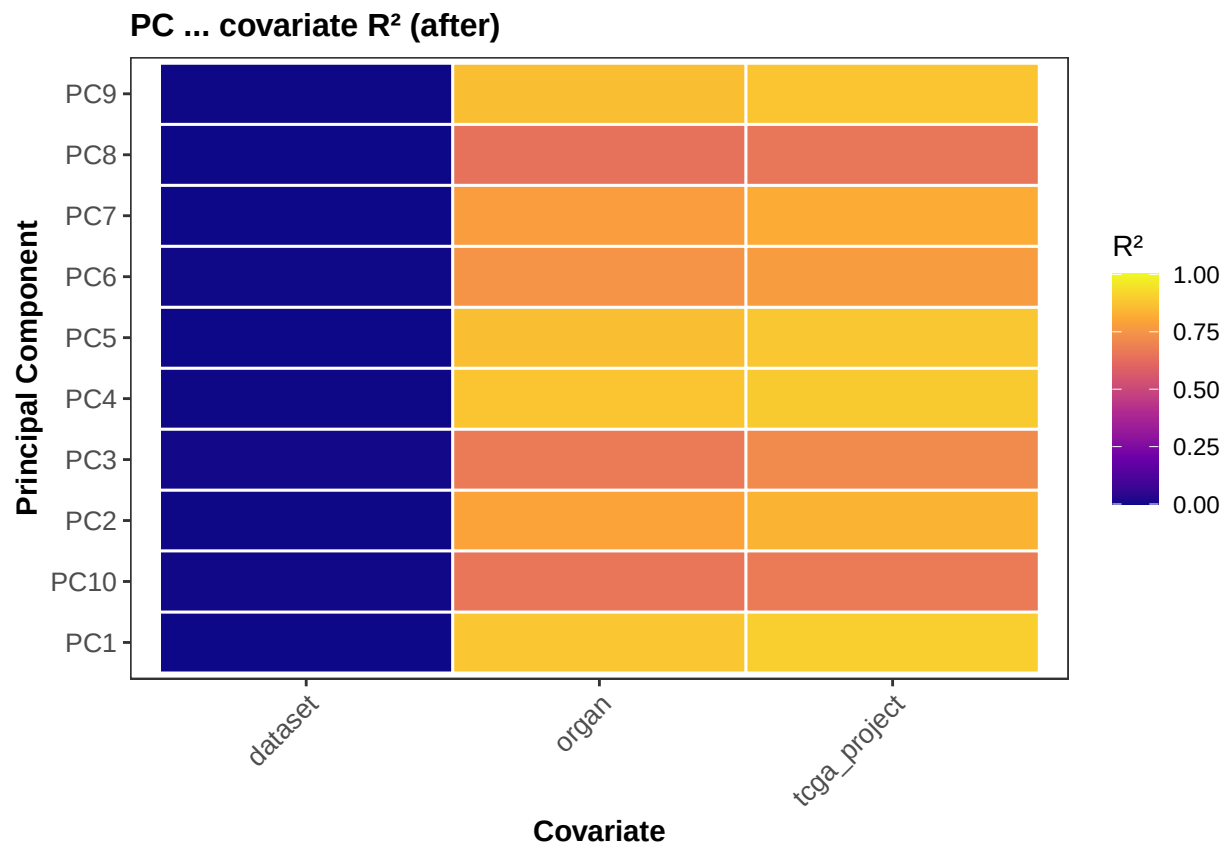


```
## [INFO] PCA AFTER batch adjustment...
```



### PCA: PC1 vs PC2 ... after Comparison





```
##
## **PC variance explained by dataset (R^2):**
##
## # A tibble: 2 x 3
##   Stage          PC1_R2    PC2_R2
##   <chr>          <dbl>    <dbl>
## 1 Before Batch Correction 0.00000411 0.116
## 2 After Batch Correction 0.000252    0.000730
## [INFO] Pipeline completed successfully. Results saved to: /home/chu25/dsmz/results/correlat.
```